OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026–27, Term-I

Project Problem Statement Submission Form

Section / Team Details

Section No.:	5
Team No.:	4
Project Title:	Multi-Agent AI Execution and Resource Management System

Team Members

Roll Number	Student Name	Signature
1	Prudvi Raj	
2	Navdeep	
3	Ashish Ignatius	

1. Abstract

Multi-Agent AI Execution and Resource Management System is a Linux-based Operating Systems project that demonstrates how multiple autonomous AI agents can communicate, coordinate, and perform tasks within a controlled execution environment. In a multi-agent AI development environment, different agents may perform tasks such as generating code, executing programs, reviewing code, and testing applications. Managing these agents safely while preventing unauthorized file access and excessive resource consumption can be challenging.

The project focuses on key Operating Systems and system programming concepts such as process creation, inter-process communication (IPC), process synchronization, file access control, CPU and memory resource management, signal handling, and process monitoring. System calls and Linux mechanisms can be used to create separate processes for AI agents, establish communication between them, control their execution, and monitor their resource usage.

The proposed solution consists of a Linux-based application in C/C++, where separate processes represent different AI agents working on a shared workspace. Named pipes (FIFOs) can be used for communication between agents, while access validation mechanisms control file operations. Signals can be used to stop or control agents that exceed resource limits, and activity logs can record agent operations and execution results.

The expected outcome is a functional demonstration of multi-agent process communication, controlled resource management, file access protection, synchronization, and process control. The project provides practical understanding of how Operating Systems can manage multiple cooperating processes while maintaining security, resource limits, and reliable execution.

2. Problem Statement

An AI development environment may contain multiple autonomous agents that perform different tasks such as generating code, executing programs, reviewing code, and testing applications. These agents may need to communicate and operate on a shared workspace at the same time. Without proper process communication, access control, and resource management mechanisms, agents may interfere with each other, access unauthorized files, or consume excessive CPU and memory resources.

This project addresses the problem of providing a controlled execution environment for multiple AI agents using Linux processes and inter-process communication mechanisms. It demonstrates how separate agent processes can communicate through named pipes (FIFOs), transfer task and execution information, and coordinate their activities on shared files.

The proposed system also validates file access before allowing an agent to perform an operation and monitors CPU and memory usage. Signals can be used to stop or control agents that exceed defined resource limits. Activity logs and backup copies of modified workspace files help maintain traceability and protect important data.

The project is relevant to Operating Systems and Systems Programming because it involves practical concepts such as process creation, IPC, process synchronization, resource management, file access control, signal handling, and process monitoring.

3. Objectives

- To understand process-based communication and resource management in a multi-agent environment.
- To create and manage separate processes representing different AI agents.
- To implement communication between agents using named pipes (FIFOs).
- To provide a restricted workspace for agent operations and validate file access.
- To monitor CPU and memory usage of agents and enforce resource limits.
- To use signals to stop or control agents exceeding resource limits.
- To maintain synchronization between agents working on shared files.
- To record agent activities, execution results, and maintain backup copies of modified workspace files.

4. Proposed Methodology

The project will be designed and implemented as a Linux-based C/C++ application that demonstrates communication, coordination, access control, and resource management between multiple AI-agent processes.

1. Agent Process Creation: The main process creates separate processes representing different AI agents.

2. Process Identification: Each agent obtains its Process ID (PID) so that the system can identify and manage individual agents.

3. Inter-Agent Communication: Named pipes (FIFOs) are created to allow agents to exchange task requests, execution information, and results.

4. Workspace Access Control: A restricted workspace is provided for agent operations, and file access is validated before an agent is allowed to perform an operation.

5. Resource Monitoring: CPU and memory usage of each agent are monitored so that excessive resource consumption can be detected.

6. Signal-Based Control: Signals are used to stop or control agents that exceed predefined CPU or memory limits.

7. Synchronization: Appropriate synchronization mechanisms are used to coordinate agents and avoid conflicts while working on shared files.

8. Logging and Backup: Agent activities, execution results, and important file modifications are automatically recorded, while backup copies of modified workspace files are maintained.

9. Execution Flow: The overall flow will be: Create Agent Processes → Identify Agents → Create FIFOs → Transfer Task Information → Validate File Access → Execute Task → Monitor Resources → Synchronize Shared Files → Log Results → Backup Modified Files → Terminate Safely.

10. Testing: The application will be executed in a Linux terminal and tested using multiple agents, different file operations, communication requests, and resource-limit conditions.

The methodology provides a practical demonstration of process management, FIFO-based IPC, resource monitoring, file access control, signal-based process control, synchronization, and Linux system programming.

5. Operating Systems Concepts / Linux APIs Used

OS Concept / Linux API / System Call	**Purpose in the Project**
Process Creation – fork()	Creates separate processes representing different AI agents.
Process Identification – getpid(), getppid()	Retrieves process IDs for identification and management of the agent hierarchy.
Inter-Process Communication – Named Pipes (FIFO)	Provides communication between independent agent processes for task and result exchange.
File Operations – open(), read(), write()	Supports controlled file read/write operations inside the restricted workspace.
Resource Monitoring – /proc filesystem	Reads Linux process information to monitor CPU, memory, and process state.
Signal Handling – signal()/sigaction()	Defines how processes respond to control and resource-management events.
Signal Control – SIGSTOP, SIGCONT, SIGTERM, SIGKILL	Pauses, resumes, safely terminates, or forcefully stops agents when required.
Process Synchronization – wait()/waitpid()	Coordinates agent completion and helps ensure proper process termination.

6. Individual Contribution

The project work is divided among the three team members so that each member has a clear technical responsibility while contributing to the final integration and testing.

Roll Number	Student Name	Individual Contribution / Responsibility
1	Prudvi Raj	**Process & IPC Module:** Implement process creation using fork(), process identification, named-pipe (FIFO) communication between AI agents, and basic process synchronization.
2	Navdeep	**Integration, Logging & Testing Module:** Integrate all agent modules, implement activity and execution-result logging and file backups, perform system testing, debug the complete application, and prepare the final demonstration/documentation.
3	Ashish Ignatius	**Resource & Security Module:** Implement restricted workspace access, file-access validation, CPU/memory monitoring through Linux /proc, resource-limit checking, and signal-based control of agents.

7. Tools / Platforms / Software Used

Tool / Platform / Software	Purpose
Linux / Ubuntu	Provides the operating environment for developing, compiling, and executing the multi-agent process management program.
C / C++	Used to implement process creation, FIFO communication, resource management, signal handling, synchronization, and file operations.
GCC / G++ Compiler	Compiles the C/C++ source code into an executable program.
POSIX Libraries	Provides APIs and functions for processes, IPC, signals, synchronization, and file operations.
Linux Terminal / Shell	Used to compile, execute, test, and monitor the multi-agent system.

8. Expected Outcome

The project is expected to produce a functional Linux-based multi-agent execution and resource management application that demonstrates how multiple AI-agent processes can communicate, coordinate, and operate safely in a shared workspace.

- The program will successfully create separate processes representing different AI agents.
- Agents will communicate and exchange task or execution information using named pipes (FIFOs).
- The system will provide a restricted workspace and validate file access before operations.
- CPU and memory usage of agents will be monitored using Linux /proc information and resource limits will be enforced.
- Signals will be used to pause, resume, terminate, or control agents when required.
- Synchronization will help prevent conflicts between agents working on shared files.
- Agent activities and execution results will be recorded automatically.
- Backup copies of modified workspace files will be maintained.
- Overall, the project will provide practical understanding of Linux process management, IPC, synchronization, resource management, file access control, signal handling, and system programming.

9. GitHub Repository

GitHub Repository Link: _____

Repository Created: ■ Yes ■ No

Faculty Name and Signature

Faculty Name: _____

Signature: _____

Remarks:

Faculty Use

Project Approved: ■ Yes ■ No

Date: _____

Faculty Signature: _____